

K8S-99: Best Practice Summary

Series: K8S — Kubernetes Monitoring | Notebook: 99 | Created: March 2026 | Last Updated: 09/24/2026

Overview

This notebook consolidates every actionable best practice for Dynatrace Kubernetes monitoring and DynaKube configuration extracted from the K8S series (notebooks 01-13). Each practice specifies the exact setting, value, priority, and category. Use this as a definitive checklist for new deployments and audits of existing environments.

Operator Version: 1.10.2 (recommended pin; 1.10.1 remains a working pin until you upgrade) | **DynaKube API:** `dynatrace.com/v1beta6` — current, use for new DynaKubes (`v1beta5` remains accepted; no rewrite required) | **Helm:** `oci://public.ecr.aws/dynatrace/dynatrace-operator --version 1.10.2` — skip 1.10.0 (pin the version you have validated in your estate)

Operator version policy (as of 08/11/2026). 1.10.2 is the recommendation — released July 30, 2026 with a published changelog. It resolves four issues: a workload/namespace **tagging-precedence regression**, a `dynatrace-webhook` `CrashLoopBackOff` under the gVisor runtime class, injected pods hanging on the OneAgent-binary download (timeout raised to 15 minutes), and metadata-enrichment rules that could not be applied being silently disregarded rather than logged.

The tagging fix is a behaviour change, not just a bug fix. Where several tagging rules match the same key, 1.10.2 applies **only the first matching rule**; earlier versions let each subsequent rule overwrite the previous one. A cluster whose enrichment depends on last-rule-wins ordering will produce different tags after the upgrade — audit your rules first. See K8S-10.

1.10.1 remains a working pin. Estates upgrade operators on their own schedule, and 1.10.1 is still the fix for the 1.10.0 defects — subject to the OpenShift-manifest caveat in K8S-09 §2. **Skip 1.10.0** — its own release notes advise skipping it (auto-update defect) and it is flagged `prerelease: true` on the [GitHub releases page](#), a signal you can verify before pinning. Do not pin below 1.4.1 in any case (CSI liveness-probe crash-loop window — K8S-09 §2).

Token currency note: Platform Tokens (`dtos16`) are the recommended choice for new tenants per Dynatrace SaaS sprint-1.337+; the Operator itself accepts platform tokens from **Operator 1.10.0** (July 15, 2026) — Classic API Tokens (`dtoc01`) continue to be accepted and remain the working path on earlier Operator versions. See K8S-02 §Prerequisites.

Operator 1.10.0 upgrade notes (July 15, 2026): the ActiveGate `TopologySpreadConstraint` default changed from `DoNotSchedule` to `ScheduleAnyway` — expect an **ActiveGate restart on upgrade** (an explicitly-set `whenUnsatisfiable` wins, so the recommended posture is to set it; see K8S-02 §8). DynaKube `v1beta3` was removed in Operator 1.9.0, and **`v1beta4` stopped being served in 1.10.0** (it was deprecated in 1.9.0) — so a `v1beta4` manifest fails to apply after this upgrade, not merely warns. `v1beta5` is still served but is itself flagged deprecated from 1.10.0. Target `v1beta6` for new DynaKubes. The `gke-autopilot.yaml` release artifact is removed — GKE Autopilot deploys via Helm. Two init containers were introduced — **`webhook-cert-generator`** and **`crd-storage-migrator`** — which is what an `Init:0/1` hang is waiting on (K8S-09 §2). 1.10.0 also adds a **CSI-to-ephemeral-volume migration mode**, which turns code-module delivery into a deliberate choice rather than a fixed default — see rule 7 and K8S-12 §2.

Table of Contents

- 1. Deployment Mode Selection
- 2. Operator Installation
- 3. DynaKube Core Configuration
- 4. ActiveGate Configuration
- 5. Namespace and Injection Control
- 6. Feature Flags and Annotations
- 7. Metadata Enrichment

- 8. [Log Monitoring](#)
- 9. [OTel Collector and Telemetry Ingest](#)
- 10. [CSI Driver Configuration](#)
- 11. [Resource Sizing](#)
- 12. [Security and Secrets](#)
- 13. [GitOps and Lifecycle](#)
- 14. [Cluster Health Alerting](#)
- 15. [Workload and Application Monitoring](#)
- 16. [Specialized Monitoring](#)
- 17. [Troubleshooting Practices](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail and Kubernetes monitoring enabled
Kubernetes Cluster	v1.24+
Helm	v3.x
Dynatrace Operator	v1.10.2 (July 30, 2026, recommended pin) via <code>oci://public.ecr.aws/dynatrace/dynatrace-operator</code> — v1.10.1 remains a working pin until you upgrade; skip 1.10.0 (see the version-policy note above)
Knowledge	K8S-01 through K8S-13

1. Deployment Mode Selection

#	Best Practice	Recommended Setting/Value	Priority	Category
1	Use <code>cloudNativeFullStack</code> for new deployments	<code>spec.oneAgent.cloudNativeFullStack: {}</code>	Critical	Deployment
2	Do NOT use <code>classicFullStack</code> for new deployments	Remove <code>spec.oneAgent.classicFullStack</code>	Critical	Deployment
3	Use <code>applicationMonitoring</code> when infra visibility is not needed	<code>spec.oneAgent.applicationMonitoring: {}</code> (CSI driver use is chosen at Operator install, not in the DynaKube)	Recommended	Deployment
4	Use <code>hostMonitoring</code> for infra-only clusters	<code>spec.oneAgent.hostMonitoring: {}</code>	Optional	Deployment
5	Omit <code>oneAgent</code> entirely for infrastructure-only monitoring alongside other APM tools	Only configure <code>spec.activeGate</code>	Recommended	Deployment

classicFullStack is legacy. It uses host-path mounts and shares a single OneAgent across all pods. `cloudNativeFullStack` injects code modules via webhook and CSI driver, enabling independent app/infra monitoring with no privileged containers for applications.

2. Operator Installation

#	Best Practice	Recommended Setting/Value	Priority	Category
6	Install operator via Helm OCI, always with an explicit <code>--version</code>	<code>helm upgrade dynatrace-operator oci://public.ecr.aws/dynatrace/dynatrace-operator --version 1.10.2 --namespace dynatrace --create-namespace --install --atomic</code>	Critical	Installation
7	Choose a code-module delivery mode deliberately	<code>csidriver.enabled: true</code> is the default recommendation; ephemeral volumes are a supported alternative from Operator 1.10.0 (see note). To migrate, use <code>csidriver.migrationMode</code> — never flip <code>csidriver.enabled: false</code> first	Critical (<i>making the choice</i>)	Installation
8	Set platform explicitly	<code>platform: "kubernetes"</code> or <code>platform: "openshift"</code>	Recommended	Installation
9	Create dedicated namespace	<code>kubectl create namespace dynatrace</code>	Critical	Installation
10	Create API token secret before applying DynaKube	<code>kubectl create secret generic dynakube --namespace dynatrace --from-literal=apiToken=<TOKEN> --from-literal=dataIngestToken=<TOKEN></code>	Critical	Installation

Rule 7 — CSI driver is now a choice, not an invariant. Through Operator 1.9.x, `csidriver.enabled: true` was effectively the only production answer, which is why earlier revisions of this checklist rated the *value* as Critical. **Operator 1.10.0 adds a CSI-to-ephemeral-volume migration mode**, so what is Critical now is *making the decision consciously* — not the specific value.

Delivery mode	Choose it when	Cost
CSI driver (<code>csidriver.enabled: true</code>)	Default. You want code modules cached per node and shared across pods, and you can run a privileged DaemonSet with a host-path socket.	An extra 5-container DaemonSet to size and monitor; a mount-storm failure mode (rules 57–59, K8S-09 §2).
Ephemeral volumes (Operator 1.10.0+)	A CSI DaemonSet is unacceptable — restrictive admission policy, a managed platform that limits CSI drivers, or a node pool where you will not run privileged workloads.	Code modules are provisioned per pod rather than shared per node, so expect more image/volume churn and slower pod starts at high density.

Migrating CSI → ephemeral (the documented direction) is a Helm-values change plus one workload-restart cycle, not an application change: enable `csidriver.migrationMode`, restart injected workloads, confirm no pod still mounts `csi.oneagent.dynatrace.com`, and only then set `csidriver.enabled: false` — pods still on CSI mounts stop working once the driver is disabled. Operator 1.10.0 was released July 15, 2026 and estates adopt it on their own schedule — **on Operator 1.9.x and earlier the CSI driver remains the only supported mode**, so `csidriver.enabled: true` stays the correct setting there. Mechanics and trade-offs: K8S-12 §2–§3.

Required Token Scopes

Operator Token: `activeGateTokenManagement.create`, `entities.read`, `settings.read`, `settings.write`, `InstallerDownload`

Data Ingest Token: `metrics.ingest`, `logs.ingest`

3. DynaKube Core Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
11	Use <code>v1beta6</code> for new DynaKubes	<code>apiVersion: dynatrace.com/v1beta6</code> . <code>v1beta5</code> still applies but is deprecated from 1.10.0 ; <code>v1beta4</code> is no longer served from 1.10.0 (apply fails); <code>v1beta3</code> removed in 1.9.0	Critical	Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
12	Set <code>apiUrl</code> to your SaaS tenant	<code>spec.apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api</code>	Critical	Configuration
13	Add control-plane tolerations to <code>OneAgent</code>	<code>tolerations: [{effect: NoSchedule, key: node-role.kubernetes.io/master, operator: Exists}, {effect: NoSchedule, key: node-role.kubernetes.io/control-plane, operator: Exists}]</code>	Recommended	Configuration
14	Set <code>nodeSelector</code> for linux-only	<code>nodeSelector: {kubernetes.io/os: linux}</code>	Recommended	Configuration
15	Set <code>OneAgent</code> resource requests and limits	<code>requests: {cpu: 100m, memory: 256Mi}, limits: {cpu: 300m, memory: 512Mi}</code>	Recommended	Configuration
16	Set <code>networkZone</code> for routing isolation	<code>spec.networkZone: production</code>	Optional	Configuration
17	Set <code>hostGroup</code> for logical grouping	<code>spec.oneAgent.hostGroup: production</code>	Optional	Configuration

Minimal Production DynaKube

```

apiVersion: dynatrace.com/v1beta6
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://ENVIRONMENT_ID.live.dynatrace.com/api
  metadataEnrichment:
    enabled: true
  oneAgent:
    cloudNativeFullStack:
      tolerations:
        - effect: NoSchedule
          key: node-role.kubernetes.io/master
          operator: Exists
        - effect: NoSchedule
          key: node-role.kubernetes.io/control-plane
          operator: Exists
  activeGate:
    capabilities:
      - kubernetes-monitoring
      - routing
      - dynatrace-api

```

4. ActiveGate Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
18	Enable <code>kubernetes-monitoring</code> capability	<code>spec.activeGate.capabilities: [kubernetes-monitoring, routing]</code>	Critical	ActiveGate
19	Add <code>dynatrace-api</code> capability	Add <code>dynatrace-api</code> to capabilities list	Recommended	ActiveGate
20	Set <code>ActiveGate</code> replicas ≥ 2 for production	<code>spec.activeGate.replicas: 2</code>	Critical	ActiveGate
21	Set <code>ActiveGate</code> resource limits	<code>limits: {cpu: 1000m, memory: 2Gi}</code> for medium clusters	Critical	ActiveGate

#	Best Practice	Recommended Setting/Value	Priority	Category
22	Add zone-aware topology spread	<code>topologySpreadConstraints: [{maxSkew: 1, topologyKey: topology.kubernetes.io/zone, whenUnsatisfiable: ScheduleAnyway}]</code>	Recommended	ActiveGate

ActiveGate Sizing Reference

Cluster Size	Nodes	CPU Limit	Memory Limit	Replicas
Small	1-10	500m	1Gi	1
Medium	10-50	1000m	2Gi	2
Large	50-100	2000m	4Gi	2
Enterprise	100+	4000m	8Gi	3+

5. Namespace and Injection Control

#	Best Practice	Recommended Setting/Value	Priority	Category
23	Use <code>namespaceSelector</code> to control injection scope	<code>spec.oneAgent.cloudNativeFullStack.namespaceSelector.matchLabels: {dynatrace-injection: enabled}</code>	Recommended	Injection
24	Do not use the default namespace for workloads	Enforce via policy	Recommended	Namespace
25	Apply consistent labels to all namespaces	<code>team</code> , <code>env</code> , <code>cost-center</code> labels	Recommended	Namespace
26	Use <code>matchExpressions</code> to exclude system namespaces	<code>operator: NotIn</code> , <code>values: [kube-system, newrelic, datadog]</code>	Recommended	Injection
27	Use pod annotation to disable injection for specific pods	<code>oneagent.dynatrace.com/inject: "false"</code>	Optional	Injection
28	Limit injected technologies when needed	<code>oneagent.dynatrace.com/technologies: "java,nodejs"</code>	Optional	Injection
29	Set ResourceQuotas on every namespace	<code>requests.cpu</code> , <code>requests.memory</code> , <code>limits.cpu</code> , <code>limits.memory</code> , <code>pods</code>	Recommended	Namespace
30	Set LimitRanges for default container limits	<code>default: {cpu: 500m, memory: 512Mi}</code> , <code>defaultRequest: {cpu: 100m, memory: 128Mi}</code>	Recommended	Namespace

6. Feature Flags and Annotations

#	Best Practice	Recommended Setting/Value	Priority	Category
31	Enable Kubernetes app detection (unofficial — not in Dynatrace docs; verify before using)	<code>feature.dynatrace.com/k8s-app-enabled: "true"</code> on DynaKube metadata	Recommended	Feature Flag

#	Best Practice	Recommended Setting/Value	Priority	Category
32	Enable version detection from K8s labels	<code>feature.dynatrace.com/label-version-detection: "true"</code>	Recommended	Feature Flag
33	Set injection failure policy to <code>fail</code> in non-prod	<code>feature.dynatrace.com/injection-failure-policy: "fail"</code>	Recommended	Feature Flag
34	Set injection failure policy to <code>silent</code> in prod	<code>feature.dynatrace.com/injection-failure-policy: "silent"</code>	Critical	Feature Flag
35	Use opt-in mode for multi-tool coexistence	<code>feature.dynatrace.com/automatic-injection: "false"</code> + <code>namespaceSelector</code>	Recommended	Feature Flag
36	Apply <code>app.kubernetes.io/version</code> label to all deployments	<code>app.kubernetes.io/version: "1.2.3"</code> in pod template labels	Recommended	Labels
37	Apply all Kubernetes recommended labels	<code>app.kubernetes.io/name</code> , <code>app.kubernetes.io/component</code> , <code>app.kubernetes.io/part-of</code>	Recommended	Labels

7. Metadata Enrichment

#	Best Practice	Recommended Setting/Value	Priority	Category
38	Enable metadata enrichment	<code>spec.metadataEnrichment.enabled: true</code>	Critical	Enrichment
39	Use settings-based enrichment (not pod annotations)	Configure rules in Settings > Cloud and virtualization > Kubernetes telemetry enrichment	Critical	Enrichment
40	Do NOT mix settings-based and pod-annotation enrichment	Use one method only	Critical	Enrichment
41	Create enrichment rules for <code>team</code> , <code>env</code> , <code>cost-center</code> labels	Metadata type: Label, Key: <code>team</code> , Prefix: <code>k8s.</code>	Recommended	Enrichment
42	Configure enrichment rules before deploying DynaKube	Rules propagate immediately when DynaKube is first applied	Recommended	Enrichment
43	Allow 45 minutes for enrichment propagation after rule changes	New/modified rules take up to 45 min on existing deployments	Recommended	Enrichment
44	Label namespaces with cost allocation metadata	<code>cost-center</code> , <code>budget-owner</code> labels on Namespace objects	Recommended	Enrichment

8. Log Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
45	Enable log monitoring with empty object	<code>spec.logMonitoring: {}</code>	Recommended	Logs
46	Configure log monitoring resources under <code>templates</code>	<code>spec.templates.logMonitoring.resources: {limits: {cpu: 500m, memory: 512Mi}}</code>	Critical	Logs
47	Never nest properties inside <code>spec.logMonitoring</code>	<code>spec.logMonitoring: {}</code> only; resources/tolerations go under <code>spec.templates.logMonitoring</code>	Critical	Logs
48	Add control-plane tolerations to log monitoring	<code>spec.templates.logMonitoring.tolerations: [{effect: NoSchedule, key: node-role.kubernetes.io/control-plane, operator: Exists}]</code>	Recommended	Logs
49	Use OpenPipeline to filter debug logs before storage	Drop <code>logLevel == "DEBUG"</code> in processing pipeline	Recommended	Logs

#	Best Practice	Recommended Setting/Value	Priority	Category
50	Route logs to separate Grail buckets by environment	OpenPipeline route rules based on <code>k8s.env</code> enriched label	Optional	Logs

9. OTEL Collector and Telemetry Ingest

#	Best Practice	Recommended Setting/Value	Priority	Category
51	Pin OTEL Collector image to a specific, verified version (example: <code>0.56.0</code> , Sept 2026 — check the releases before installing)	<code>spec.templates.otelCollector.imageRef.tag: "0.56.0"</code>	Critical	OTel
52	Never use <code>latest</code> tag for OTEL Collector	Always specify explicit version tag	Critical	OTel
53	Set OTEL Collector resource limits	<code>limits: {cpu: 500m, memory: 512Mi}</code> for staging; <code>{cpu: 1000m, memory: 1Gi}</code> for production	Recommended	OTel
54	Configure <code>telemetryIngest</code> with required protocols	<code>spec.telemetryIngest.protocols: [otlp]</code> (add <code>statsd</code> , <code>jaeger</code> , <code>zipkin</code> as needed)	Recommended	OTel
55	Use StatsD via OTEL Collector on K8s (not OneAgent StatsD daemon)	Deploy OTEL Collector with StatsD receiver; OneAgent StatsD is not available on K8s	Critical	OTel
56	Deploy StatsD OTEL Collector in a dedicated <code>monitoring</code> namespace	Cluster-wide Deployment; apps reference via FQDN <code>otel-collector-statsd.monitoring.svc.cluster.local:8125</code>	Recommended	OTel

10. CSI Driver Configuration

#	Best Practice	Recommended Setting/Value	Priority	Category
57	Decide CSI vs. ephemeral volumes, then set it explicitly	<code>csidriver.enabled: true</code> for the CSI path (the default recommendation); ephemeral volumes are supported from Operator 1.10.0 — see rule 7	Critical (<i>making the choice</i>)	CSI
58	Set resource limits on provisioner container	<code>provisioner.resources.limits: {cpu: 500m, memory: 256Mi}</code>	Critical	CSI
59	Set resource limits on all 5 CSI containers	<code>csiInit</code> , <code>server</code> , <code>provisioner</code> , <code>registrar</code> , <code>livenessprobe</code>	Recommended	CSI

CSI Driver Container Resource Reference

Container	CPU Request	CPU Limit	Memory Request	Memory Limit
<code>csiInit</code>	50m	100m	100Mi	128Mi
<code>server</code>	50m	100m	100Mi	128Mi
<code>provisioner</code>	300m	500m	100Mi	256Mi
<code>registrar</code>	20m	50m	30Mi	64Mi
<code>livenessprobe</code>	20m	50m	30Mi	64Mi

Warning: Default Helm `values.yaml` may be missing limits for the `provisioner` container. Always add them explicitly.

Rules 58–59 apply only on the CSI path. If you chose ephemeral volumes (Operator 1.10.0+, see rule 7) there is no CSI DaemonSet to size, and this entire section is not applicable — the corresponding work moves to per-pod volume provisioning and

pod-start latency at high density. On Operator 1.9.x and earlier the CSI path is the only supported mode, so these rules always apply.

11. Resource Sizing

#	Best Practice	Recommended Setting/Value	Priority	Category
60	Set OneAgent resources for cloudNativeFullStack	<code>requests: {cpu: 100m, memory: 256Mi}, limits: {cpu: 300m, memory: 512Mi}</code>	Recommended	Sizing
61	Size ActiveGate by cluster node count	See ActiveGate Sizing Reference in Section 4	Critical	Sizing
62	Size OTel Collector by telemetry volume	Low: 200m/256Mi, Medium: 500m/512Mi, High: 1000m/1Gi, Very High: 2000m/2Gi	Recommended	Sizing
63	Size Log Monitoring by daily log volume	Low (<1GB): 200m/256Mi, Medium (1-10GB): 500m/512Mi, High (10-50GB): 1000m/1Gi	Recommended	Sizing
64	Monitor OneAgent memory by absolute usage (no limits by default)	OneAgent runs without resource limits; use <code>dt.kubernetes.container.memory_working_set</code> with <code>matchesValue(k8s.container.name, "dynatrace-oneagent")</code>	Recommended	Sizing

12. Security and Secrets

#	Best Practice	Recommended Setting/Value	Priority	Category
65	Never store API tokens in Git	Use External Secrets Operator, Sealed Secrets, or SOPS	Critical	Security
66	Use External Secrets Operator for token management	<code>ExternalSecret</code> CR referencing Vault, AWS Secrets Manager, or GCP Secret Manager	Recommended	Security
67	Apply NetworkPolicies allowing Dynatrace egress on port 443	<code>egress: [{to: [{ipBlock: {cidr: 0.0.0.0/0}}], ports: [{protocol: TCP, port: 443}]}]</code>	Recommended	Security
68	Configure proxy via DynaKube spec (not env vars)	<code>spec.proxy.value: https://proxy.example.com:8080</code> or <code>spec.proxy.valueFrom.secretKeyRef</code>	Recommended	Security
69	Apply RBAC per namespace aligned with Dynatrace boundaries	<code>K8s namespace-admin</code> = full namespace visibility in Dynatrace	Recommended	Security

13. GitOps and Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Category
70	Manage DynaKube via GitOps (ArgoCD or Flux)	Store DynaKube YAML in Git, apply via GitOps controller	Recommended	Lifecycle
71	Use Kustomize overlays for environment-specific config	<code>base/dynakube.yaml</code> + <code>overlays/{dev,staging,prod}/</code> patches	Recommended	Lifecycle
72	Pin an exact operator chart version in ArgoCD/Flux	<code>targetRevision: 1.10.2</code> (ArgoCD) or <code>version: "1.10.2"</code> (Flux) — prefer an exact pin over a <code>1.10.x</code> range, because an operator upgrade can change chart <i>defaults</i> with no Git commit to review (K8S-02 §8). Never pin below 1.4.1; skip 1.10.0	Critical	Lifecycle
73	Set <code>selfHeal: true</code> in ArgoCD for drift correction	<code>syncPolicy.automated.selfHeal: true</code>	Recommended	Lifecycle
74	Set <code>prune: false</code> for DynaKube in ArgoCD	Prevent accidental DynaKube deletion on Git removal	Critical	Lifecycle

#	Best Practice	Recommended Setting/Value	Priority	Category
75	Use Flux dependsOn to order operator before DynaKube	<code>dependsOn: [{name: dynatrace-operator}]</code>	Recommended	Lifecycle
76	Use ArgoCD sync waves for deployment ordering	Wave 0: Namespace, Wave 1: Operator, Wave 2: DynaKube	Recommended	Lifecycle
77	Use <code>helm upgrade --reuse-values</code> for operator upgrades — and diff the rendered chart	Preserves <i>your</i> values, but not changed chart <i>defaults</i> for values you never set. Render and diff before upgrading (<code>helm template ... > new.yaml</code> vs. <code>helm get manifest</code>) — K8S-02 §8	Recommended	Lifecycle
78	Deploy to dev -> staging -> prod-canary -> prod	Progressive rollout strategy	Recommended	Lifecycle

14. Cluster Health Alerting

#	Best Practice	Recommended Setting/Value	Priority	Category
79	Alert on Node NotReady	Node condition != Ready for >5 min = Critical	Critical	Alerting
80	Alert on high node CPU	CPU > 85% for 15 min = Warning	Recommended	Alerting
81	Alert on high node memory	Memory > 90% for 10 min = Critical	Critical	Alerting
82	Alert on disk pressure	Disk > 85% = Warning	Recommended	Alerting
83	Alert on OOMKilled events	Any OOMKilled event = Warning	Critical	Alerting
84	Alert on CrashLoopBackOff	Any CrashLoopBackOff = Warning	Recommended	Alerting
85	Alert on FailedScheduling	Events > 5 in 10 min = Warning	Recommended	Alerting
86	Alert on volume mount failures	Any FailedMount or FailedAttachVolume = Critical	Critical	Alerting
87	Monitor Dynatrace component health (OA, AG, CSI)	Track restarts, OOMKills, and data gaps for <code>dynatrace-oneagent</code> and <code>activegate</code> containers	Critical	Alerting
88	Target CPU utilization >40% avg for cost efficiency	Right-size if avg CPU usage <40%	Recommended	Cost
89	Target memory utilization >50% avg for cost efficiency	Right-size if avg memory usage <50%	Recommended	Cost

15. Workload and Application Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
90	Alert on container CPU usage > 85% sustained	Monitor <code>dt.kubernetes.container.cpu_usage</code> vs limits	Recommended	Workload
91	Alert on container memory usage > 90%	Monitor <code>dt.kubernetes.container.memory_working_set</code> vs limits	Critical	Workload
92	Monitor CPU throttling	<code>dt.containers.cpu.throttled_time > 0</code> indicates under-provisioned CPU limits	Recommended	Workload
93	Track service P99 latency and error rate	SLO: P99 < 500ms, error rate < 0.1%	Recommended	Workload
94	Filter on <code>isNotNull(field)</code> before aggregating optional fields	Prevent null group-by keys in DQL	Recommended	Workload

#	Best Practice	Recommended Setting/Value	Priority	Category
95	Use <code>arrayAvg()</code> before filtering/sorting timeseries results	<code>timeseries</code> returns arrays; convert to scalar first	Critical	DQL

16. Specialized Monitoring

#	Best Practice	Recommended Setting/Value	Priority	Category
96	Instrument NGINX Ingress via ConfigMap <code>main-snippet</code>	<code>load_module /opt/dynatrace/oneagent-paas/agent/bin/current/linux-musl-x86-64/liboneagentnginx.so;</code>	Recommended	NGINX
97	NGINX instrumentation requires x86-64 and OneAgent 1.227+	No ARM support; pod name must contain <code>ingress-nginx-</code>	Recommended	NGINX
98	Enable Prometheus scraping for custom metrics	Annotate pods with <code>metrics.dynatrace.com/scrape: "true"</code> , <code>/port</code> , <code>/path</code>	Recommended	Prometheus
99	ActiveGate must be in-cluster for annotation-based Prometheus scraping	External ActiveGate cannot reach pod endpoints	Critical	Prometheus
100	Use Kpow + Prometheus scraping for Kafka consumer lag monitoring	<code>PROMETHEUS_EGRESS: "true"</code> in Kpow config; scrape annotations on Kpow pods	Optional	Kafka

17. Troubleshooting Practices

#	Best Practice	Recommended Setting/Value	Priority	Category
101	Check DynaKube status first	<code>kubectl -n dynatrace get dynakube</code> — expect <code>Running</code> phase	Critical	Troubleshooting
102	Verify all Dynatrace pods are running	<code>kubectl -n dynatrace get pods</code> — no <code>CrashLoopBackOff</code> or <code>Error</code>	Critical	Troubleshooting
103	Check OneAgent connectivity to tenant	<code>kubectl -n dynatrace exec <oneagent-pod> -c oneagent -- curl -v https://<tenant>.live.dynatrace.com/api/v1/time</code>	Recommended	Troubleshooting
104	Check namespace labels when injection fails	<code>kubectl get namespace <ns> -o jsonpath='{.metadata.labels}'</code> must match DynaKube <code>namespaceSelector</code>	Recommended	Troubleshooting
105	Verify init container present for injected pods	<code>kubectl get pod <pod> -o jsonpath='{.spec.initContainers[*].name}'</code>	Recommended	Troubleshooting
106	Check webhook registration	<code>kubectl get mutatingwebhookconfigurations</code> — Dynatrace webhook must exist	Recommended	Troubleshooting
107	Collect support bundle for escalation	DynaKube YAML, pod YAML, events, operator logs, OneAgent logs in a <code>tar.gz</code>	Optional	Troubleshooting
108	Use DQL to detect Dynatrace component failures across fleet	Query <code>events</code> for <code>dynatrace</code> + <code>Failed/OOMKilled/BackOff</code>	Recommended	Troubleshooting

#	Best Practice	Recommended Setting/Value	Priority	Category
109	Detect metric data gaps from null buckets	<code>timeseries ... = avg(dt.host.cpu.usage)</code> then compare <code>arraySize(arrayRemoveNulls(...))</code> to <code>arraySize(...)</code> — <code>arraySize</code> counts empty buckets and <code>arrayMin</code> skips them	Recommended	Troubleshooting

Summary

This notebook contains **109 actionable best practices** across 17 categories for Dynatrace Kubernetes monitoring:

Category	Count	Key Takeaway
Deployment Mode	5	Use <code>cloudNativeFullStack</code> ; never <code>classicFullStack</code> for new deployments
Operator Installation	5	Helm OCI install with CSI driver enabled
DynaKube Core	7	v1beta5/v1beta6, tolerations, resource limits
ActiveGate	5	<code>kubernetes-monitoring</code> + <code>routing</code> , 2+ replicas in prod
Namespace/Injection	8	<code>namespaceSelector</code> for scope control, ResourceQuotas on every namespace
Feature Flags	7	Version detection, K8s app detection, injection failure policy
Metadata Enrichment	7	Settings-based only, never mix with pod annotations
Log Monitoring	6	Empty <code>logMonitoring: {}</code> , resources under <code>templates</code>
OTel/Telemetry Ingest	6	Pin version, StatsD via OTel Collector only
CSI Driver	3	Always set provisioner limits
Resource Sizing	5	Size by cluster/volume tier
Security	5	Never store tokens in Git, use ESO
GitOps/Lifecycle	9	Pin versions, progressive rollouts, prune: false for DynaKube
Cluster Alerting	11	Node health, OOMKill, CrashLoop, Dynatrace component health
Workload Monitoring	6	CPU throttling, P99 SLOs, <code>arrayAvg()</code> for timeseries
Specialized Monitoring	5	NGINX via ConfigMap, in-cluster AG for Prometheus
Troubleshooting	9	DynaKube status, connectivity, DQL-based diagnostics

References

- [DynaKube parameters \(DT docs\)](#)
- [DynaKube feature flags \(DT docs\)](#)
- [Set up Dynatrace on Kubernetes \(DT docs\)](#)
- [How Dynatrace Kubernetes monitoring works \(DT docs\)](#)
- [Quickstart for K8s setup \(DT docs\)](#)
- [Full observability deployment \(DT docs\)](#)
- [Kubernetes log monitoring \(DT docs\)](#)
- [Metadata enrichment \(DT docs\)](#)
- [Kubernetes app \(DT docs\)](#)
- [Operator + DynaKube troubleshooting \(DT docs\)](#)
- [Dynatrace Operator Helm chart \(Dynatrace GitHub\)](#)
- [Dynatrace Operator releases \(Dynatrace GitHub\)](#)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.